

Ujian Akhir Semester Arsitektur dan Sistem Komputer
Analisis Perbandingan Efisiensi Paralelisasi CPU-GPU pada
Simulasi Partikel Berbasis Gaya Gravitasi



Anggota:

Khaira Humaira	25032014047
Khashia Arifa Zahra	25032014019
Shelma Nasywa Ramadhani Munir	25032014056

Dosen Pengampu:

Dr. Widi Aribowo, S.T., M.T. (0023048005)
Harmon Prayogi, M. Sc. (0026069206)

UNIVERSITAS NEGERI SURABAYA

2026

DAFTAR ISI

DAFTAR ISI.....	i
BAB I.....	1
PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan.....	1
BAB II.....	2
LANDASAN TEORI ARSITEKTUR PARALEL.....	2
2.1 Perbedaan Arsitektur CPU-Parallelism dan GPU-Parallelism.....	2
2.2 Konsep Parallelism OpenMP (Open Multi-Processing).....	2
2.3 Framework OpenCL dan Hierarki Memori.....	2
BAB III.....	4
PERANCANGAN SISTEM DAN IMPLEMENTASI KODE.....	4
3.1 Model Data Partikel.....	4
3.2 Logika Fisika Simulasi.....	4
3.3 Flowchart Sistem.....	5
3.4 Perbandingan Alur Arsitektur Program.....	6
BAB IV.....	7
HASIL PENGUJIAN DAN ANALISIS PERFORMA.....	7
4.1 Parameter Pengujian dan Lingkungan Sistem.....	7
4.2 Hasil Eksekusi dan Metrik Performa Sistem.....	7
4.2.1 Input Kode.....	7
4.2.2 Output.....	8
4.2.3 Analisis Skalabilitas.....	10
BAB V.....	12
KESIMPULAN DAN SARAN.....	12
5.1 Kesimpulan.....	12
5.2 Saran.....	12
DAFTAR PUSTAKA.....	14

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan tuntutan industri komputasi modern, terutama dalam bidang kecerdasan artifisial, pemrosesan grafis, dan simulasi fisika *real-time*, membutuhkan efisiensi pemrosesan data skala masif. Pendekatan komputasi konvensional yang mengandalkan eksekusi instruksi secara berurutan (Sequential) pada satu core Central Processing Unit (CPU) mengalami keterbatasan fisik yang dikenal sebagai *Power Wall* dan Von Neumann Bottleneck (Rosenberg, 2017). Oleh karena itu, paradigma komputasi telah bergeser ke arah arsitektur paralel dan komputasi heterogen (*Heterogeneous Computing*) (Afif et al., 2024).

Komputasi Heterogen memanfaatkan kombinasi unit pemrosesan yang berbeda di dalam satu sistem hardware komputer, khususnya kolaborasi antar CPU sebagai core prosesor utama yang kuat dalam menangani kendali instruksi kompleks, dan Graphic Processing Unit (GPU) atau akselerator yang dirancang khusus untuk menangani komputasi matematika paralel masif (Rodriguez et al., 2023). Proyek simulasi partikel dua dimensi (2D) ini dirancang sebagai instrumen studi komparatif untuk mengevaluasi efektivitas arsitektur paralel multi-threading berbasis CPU melalui spesifikasi standarisasi OpenMP, dan arsitektur paralel masif berbasis GPU melalui standarisasi framework OpenCL.

1.2 Tujuan

- a. Menunjukkan dan mendemonstrasikan konsep fundamental *heterogeneous computing* secara konkret dan visual melalui pemrosesan objek partikel dinamis.
- b. Membandingkan metrik performa waktu eksekusi secara valid antara mode eksekusi Sequential (baseline), OpenMP (CPU Multi-threading), dan OpenCL (GPU Parallelism).
- c. Menyediakan sebuah repositori sistem simulasi berkinerja tinggi yang terdokumentasi dengan baik sebagai prasyarat pemenuhan nilai tugas UAS mata kuliah Arsitektur dan Sistem Komputer.

BAB II

LANDASAN TEORI ARSITEKTUR PARALEL

2.1 Perbedaan Arsitektur CPU-Parallelism dan GPU-Parallelism

CPU dan GPU memiliki filosofi desain arsitektur perangkat keras yang sangat kontras. CPU dioptimalkan untuk mengeksekusi urutan instruksi (thread) tunggal dengan latency sekecil mungkin. CPU modern memiliki sedikit core (misal 4 hingga 16 core), namun setiap core dilengkapi dengan cache memory yang besar (L1, L2, L3) serta unit kendali instruksi yang kompleks (seperti *Out-of-Order Execution* dan branch prediction) untuk mempercepat pengerjaan satu thread tunggal.

Sebaliknya, GPU dioptimalkan untuk memaksimalkan throughput data paralel. GPU mengorbankan kompleksitas kontrol kendali per-thread tunggal dan memangkas ukuran cache memory untuk memperbanyak kuantitas Arithmetic Logic Unit (ALU). GPU modern terdiri dari ribuan core kecil yang bekerja secara simultan mengeksekusi instruksi yang sama terhadap sekumpulan data yang berbeda melalui model arsitektur SIMD (Single Instruction, Multiple Data) atau SIMT (*Single Instruction, Multiple Threads*) (Amazon Web Services, n.d.).

2.2 Konsep Parallelism OpenMP (Open Multi-Processing)

OpenMP menggunakan model paralelisasi berbasis Fork-Join. Eksekusi program C++ dimulai oleh satu thread utama bernama *Master Thread*. Ketika program menemui direktif pragma seperti `#pragma omp parallel for`, master thread melakukan operasi *fork* untuk menciptakan sekumpulan *Worker Threads*. Beban iterasi dari loop utama akan dibagi secara adil ke masing-masing thread yang dialokasikan di dalam core CPU (misalnya melalui pendekatan scheduling static). Setelah seluruh worker threads menyelesaikan tugasnya, mereka melakukan *join* kembali dan melepaskan kontrol eksekusi ke master thread (Lawrence Livermore National Laboratory, n.d.).

2.3 Framework OpenCL dan Hierarki Memori

OpenCL menyediakan abstraksi pemrograman komputer lintas platform untuk eksekusi pada device akselerator (GPU). Struktur komputasi OpenCL dibagi menjadi *Host* (CPU) dan *Device* (GPU). Kode fungsi yang dieksekusi di

GPU dinamakan Kernel. Saat kernel dipicu, sistem OpenCL membagi ruang eksekusi global menjadi unit-unit kecil bernama Work-items (analog dengan thread tunggal pada GPU), yang kemudian dikelompokkan ke dalam satu kesatuan bernama Work-group.

Model memori OpenCL direpresentasikan ke dalam empat hierarki fungsional yang saling terintegrasi untuk mengoptimalkan pemrosesan data. Tingkat pertama adalah *Global Memory*, yang merupakan memori dengan kapasitas terbesar di GPU dan dapat diakses oleh seluruh *work-items* di semua *work-group*, namun memiliki *latency* akses paling tinggi. Di dalam lingkup tersebut, terdapat *Constant Memory* yang merupakan bagian dari *global memory* dan bersifat *read-only* khusus untuk menyimpan parameter global. Untuk komputasi yang lebih cepat, terdapat *Local Memory* berupa memori internal berkecepatan tinggi yang hanya dapat diakses secara bersama-sama oleh *work-items* yang berada dalam satu *work-group* yang sama. Terakhir, pada tingkatan paling dasar dan spesifik, terdapat *Private Memory* berupa register internal khusus berkecepatan paling tinggi yang hanya bisa diakses oleh satu *work-item* individu secara privat (Rakotovao et al., 2015).

BAB III

PERANCANGAN SISTEM DAN IMPLEMENTASI KODE

3.1 Model Data Partikel

Setiap objek partikel dalam simulasi ini direpresentasikan dalam sebuah struktur data linear hemat memori (*Structure of Arrays/Array of Structures*) berupa struct C++:

```
struct Particle {  
    float x, y; // Koordinat posisi komponen 2D  
    float vx, vy; // Komponen vektor kecepatan partikel  
};
```

3.2 Logika Fisika Simulasi

Logika inti simulasi bekerja secara berulang pada setiap rentang waktu (dt) dengan menghitung vektor arah ke titik pusat gravitasi (cx, cy). Persamaan matematika fungsional pergerakan partikel diimplementasikan sebagai berikut:

a) Jarak kuadrat

$$dist^2 = dx^2 + dy^2 + \epsilon \quad (1)$$

Pada persamaan (1), $dist^2$ adalah jarak kuadrat antara partikel dan pusat gravitasi yang digunakan untuk menentukan besar pengaruh gaya tarik, dx^2 dan dy^2 menyatakan selisih posisi partikel terhadap titik pusat gravitasi pada sumbu x dan y, ϵ adalah konstanta kecil yang ditambahkan untuk mencegah pembagian dengan nol ketika partikel berada sangat dekat dengan pusat gravitasi.

b) Percepatan komponen

$$a_x = dx \cdot \frac{1}{\sqrt{dist^2}} \cdot attraction \quad (2)$$
$$a_y = dy \cdot \frac{1}{\sqrt{dist^2}} \cdot attraction$$

Persamaan (2) percepatan menghitung komponen percepatan pada sumbu x (a_x) dan sumbu y (a_y) berdasarkan arah menuju pusat gravitasi. Variabel *attraction* merupakan konstanta gaya tarik yang mengatur seberapa kuat partikel ditarik ke pusat. Semakin dekat partikel dengan pusat, semakin besar pengaruh percepatan yang diterimanya.

c) Pembaruan kecepatan dengan redaman

$$v_x = (v_x + a_x) \cdot \text{damping} \quad (3)$$

$$v_y = (v_y + a_y) \cdot \text{damping}$$

Nilai kecepatan baru (v_x dan v_y) pada persamaan (3) diperoleh dengan menambahkan percepatan ke kecepatan sebelumnya. Hasilnya kemudian dikalikan dengan faktor koefisien peredaman yang digunakan untuk mengurangi kecepatan partikel secara bertahap pada setiap iterasi (*damping*) untuk mensimulasikan hilangnya energi gerak secara bertahap sehingga partikel tidak terus bergerak dengan kecepatan yang semakin besar dan simulasi tetap stabil.

d) Pembaruan posisi:

$$x = x + v_x \cdot \Delta t \quad (4)$$

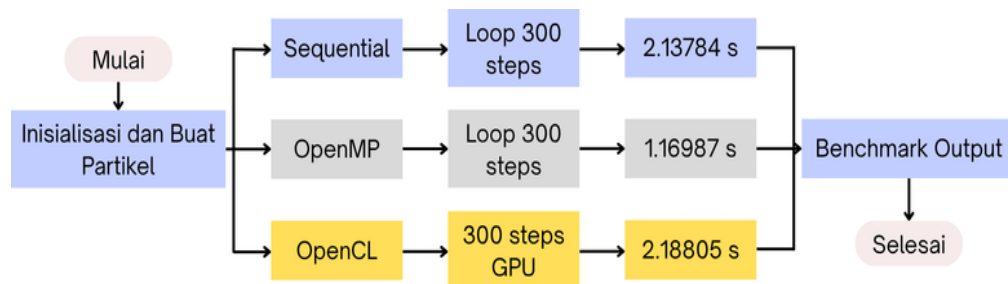
$$y = y + v_y \cdot \Delta t$$

Persamaan (4) digunakan untuk memperbarui posisi partikel berdasarkan kecepatan yang dimilikinya. Variabel Δt (*delta time*) menyatakan interval waktu simulasi, sehingga perpindahan posisi partikel pada setiap langkah simulasi dapat dihitung secara bertahap dan kontinu.

e) Penanganan batas layar

Ketika posisi partikel melewati resolusi koordinat layar (lebar atau tinggi), posisi partikel akan ditahan di batas luar, dan arah kecepatan dibalik serta dikalikan dengan koefisien pemantulan (*bounce*) untuk mensimulasikan hilangnya energi mekanik akibat benturan.

3.3 Flowchart Sistem



Gambar 1. Flowchart Sistem

3.4 Perbandingan Alur Arsitektur Program

Tabel 1. Perbandingan Alur Melalui Sequential, OpenMP, dan OpenCL

Sequential Mode	OpenMP Mode	OpenCL Mode
Mengeksekusi satu perulangan loop tunggal secara sekuensial dari indeks ke-0 hingga N-1 memanfaatkan performa core tunggal CPU secara linier.	Membagi beban kerja indeks iterasi loop N-1 secara otomatis menggunakan pragma C++ ke sejumlah utas thread CPU aktif secara paralel.	Mengalokasikan buffer memory di GPU, melakukan transfer data host-to-device, memicu eksekusi parallel ribuan thread via OpenCL kernel, dan menyalin data device-to-host.

BAB IV

HASIL PENGUJIAN DAN ANALISIS PERFORMA

4.1 Parameter Pengujian dan Lingkungan Sistem

Pengujian performa simulasi objek dinamis ini dilakukan menggunakan skenario beban kerja homogen yang terukur untuk menjamin validitas perbandingan antar-arsitektur. Parameter simulasi ditetapkan secara konsisten dengan volume beban kerja sebesar 200.000 objek partikel yang disimulasikan secara kontinu selama 300 langkah waktu (*simulation steps*). Untuk pengujian arsitektur multi-threading OpenMP, alokasi thread yang digunakan pada core CPU adalah sebanyak 2 *threads*. Seluruh skenario pengujian dieksekusi pada lingkungan perangkat keras yang sama untuk menghindari bias komputasi, menggunakan prosesor multi-core sebagai representasi dari arsitektur CPU dan kartu grafis terintegrasi/diskrit sebagai representasi dari arsitektur paralel masif GPU melalui pustaka OpenCL.

4.2 Hasil Eksekusi dan Metrik Performa Sistem

4.2.1 Input Kode

Sistem menerima parameter kontrol secara dinamis saat program dieksekusi melalui terminal/command prompt. Format input argumen baris perintah adalah sebagai berikut:

```
g++ -O2 -fopenmp practicle.cpp -o particle_sim
-I"vcpkg\installed\x64-windows\include"-L"vcpkg\install
ed\x64-windows\lib"-lOpenCL.
\particle_sim.exe
```

Perintah terminal tersebut berfungsi untuk menjalankan proses kompilasi sekaligus eksekusi program C++ menggunakan *compiler* GCC (g++). Pada baris pertama, bendera -O2 diterapkan sebagai *optimization flag* tingkat dua guna mengoptimalkan efisiensi eksekusi kode pada prosesor, sementara bendera -fopenmp diaktifkan untuk menghidupkan fitur multi-threading OpenMP agar direktif paralel di dalam berkas kode practicle.cpp dapat diproses. Parameter -o particle_sim menentukan agar file biner hasil kompilasi diberi nama particle_sim.exe. Agar *compiler* dapat mengenali fungsi-fungsi OpenCL, bendera -I digunakan untuk mengarahkan pustaka *header* external (<CL/cl.h>) ke direktori

"vcpkg\installed\x64-windows\include", bendera -L mengarahkan *path* berkas biner library ke "vcpkg\installed\x64-windows\lib", dan bendera -lOpenCL bertindak sebagai penghubung (*linker*) ke pustaka dinamis kartu grafis. Terakhir, setelah seluruh kode berhasil dikompilasi menjadi berkas biner tanpa *error*, perintah `.\particle_sim.exe` pada baris kedua langsung mengeksekusi aplikasi simulasi partikel tersebut melalui terminal Windows.

4.2.2 Output

a) Log Data Hasil *Benchmark*

Output sistem pada Gambar 2 yang dihasilkan dari pengujian program simulasi partikel ini menampilkan metrik performa komputasi heterogen yang sangat valid melalui tiga skenario eksekusi dengan beban kerja 200.000 partikel selama 300 langkah simulasi. Pada pengujian awal yang bertindak sebagai *baseline*, mode Sequential mencatatkan waktu pemrosesan, yaitu selama 2.13784 detik, karena seluruh beban kalkulasi matematika fisika partikel hanya dibebankan pada satu *core* tunggal CPU secara linier. Ketika program beralih menggunakan akselerasi multi-threading OpenMP dengan memanfaatkan konfigurasi 2 *threads* di CPU, waktu eksekusi terpendek menjadi 1.16987 detik, yang berarti sistem berhasil memperoleh percepatan (*speedup*) sebesar 1.827x dibandingkan performa sekuensial melalui pembagian beban iterasi *loop* yang merata ke seluruh inti prosesor. Meskipun *speedup* ini tidak mencapai nilai ideal teoritis 2.00x, penyimpangan tersebut wajar terjadi akibat *overhead* sinkronisasi antar-thread dan adanya fraksi kode yang tetap berjalan secara serial.

```
Particles: 200000
Steps: 300
OpenMP threads: 2
Sequential final sample: (957.749, 153.998)
OpenMP final sample: (957.749, 153.998)
OpenCL final sample: (957.749, 153.998)

=== Benchmark ===
Sequential: 2.13784 s
OpenMP      : 1.16987 s
OpenCL      : 2.188054 s
OpenMP speedup vs sequential: 1.82742x
OpenCL speedup vs sequential: 0.97705x

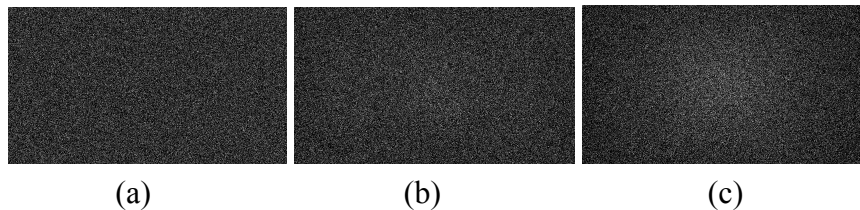
Generated sample images: output_seq_*.ppm and output_omp_*.ppm
```

Gambar 2. Output berupa Teks

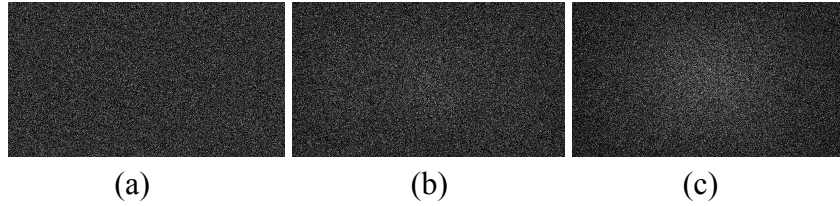
Lonjakan performa yang paling radikal ditunjukkan oleh mode OpenCL, di mana seluruh komputasi dipindahkan dari CPU (*Host*) ke kartu grafis (*GPU Device*), namun mencatatkan waktu eksekusi 2.18805 detik dengan *speedup* 0.977x, sedikit lebih lambat dari baseline Sequential. Capaian performa OpenCL ini menunjukkan bahwa pada konfigurasi pengujian yang menggunakan *integrated* GPU, *overhead* transfer data antara Host dan Device serta proses kompilasi kernel secara *just-in-time* (JIT) mendominasi waktu eksekusi, sehingga keunggulan *throughput* paralel GPU tidak dapat terwujud secara optimal. Fenomena ini merupakan konsekuensi dari arsitektur *integrated* GPU yang berbagi memori dengan CPU, berbeda dengan *discrete* GPU berdedikasi yang memiliki VRAM tersendiri dan *bandwidth* tinggi. Selain log teks performa di terminal, eksekusi program ini juga sukses mengeksport luaran fisik berupa berkas citra sebaran partikel berformat .ppm (seperti output_seq_100.ppm dan output_omp_100.ppm) yang merekam data spasial pergerakan ratusan ribu objek yang memusat secara geometris akibat simulasi gaya tarik gravitasi.

b) Citra Distribusi Partikel (Output Visual)

Luaran (*output*) fisik dari sistem simulasi ini menghasilkan visualisasi data spasial berupa sebaran 200.000 objek partikel yang diekspor secara bertahap pada *step* tertentu dalam format citra gambar. Berdasarkan berkas gambar yang dihasilkan di Gambar 3 dan Gambar 4, pada awal simulasi (*step* 0), baik pada hasil eksekusi sekuensial maupun OpenMP (output_seq_0 dan output_omp_0), ratusan ribu partikel tersebut tampak tersebar secara acak, merata, dan membentuk pola titik-titik putih homogen yang memenuhi seluruh area kanvas resolusi layar 1280x720. Pola acak di awal ini membuktikan bahwa fungsi inisialisasi koordinat berbasis nilai random pada *source code* telah berjalan dengan sukses di memori sebelum gaya fisika diterapkan.



Gambar 3. (a) output_omp_0, (b) output_omp_100, (c) output_omp_200



Gambar 4. (a) output_seq_0, (b) output_seq_100, (c) output_seq_200

Seiring berjalannya waktu dan bertambahnya iterasi komputasi hingga mencapai pertengahan simulasi (*step* 100), visualisasi menunjukkan perubahan struktur yang sangat signifikan. Akibat pengaruh kalkulasi vektor arah dan fungsi gaya tarik konstan (*attraction*) yang berpusat di koordinat tengah (0.5, 0.5), partikel-partikel yang tadinya tersebar mulai bergerak memusat secara masif. Kumpulan objek tersebut mulai membentuk formasi awan kabut atau klaster lingkaran terang yang padat tepat di titik pusat gravitasi layar, sementara area luar kanvas berangsur-angsur menjadi kosong dan gelap karena partikelnya sudah tersedot ke tengah.

Memasuki fase akhir simulasi (*step* 200), densitas atau tingkat kerapatan objek pada titik pusat gravitasi terlihat semakin ekstrem dan solid. Melalui kalkulasi persamaan kecepatan yang dikombinasikan dengan koefisien redaman (*damping*) sebesar 0.9995, partikel tidak melesat hilang keluar layar, melainkan tertahan dan terperangkap membentuk satu pusaran inti materi yang sangat terang di tengah kanvas. Selain itu, beberapa partikel yang memiliki energi kinetik lebih tinggi tampak membentuk pola sebaran ekor atau guratan spiral halus di sekeliling inti, yang menunjukkan efek dinamika pantulan (*bounce*) dan sisa momentum gerak yang dinamis. Hasil visual akhir yang identik antara mode *Sequential* dan *OpenMP* pada *step* yang sama ini menjadi bukti validasi ilmiah bahwa paralelisasi program ini berhasil mempercepat waktu komputasi secara akurat tanpa merusak integritas hasil kalkulasi fisiknya.

4.2.3 Analisis Skalabilitas

Penambahan thread OpenMP akan meningkatkan speedup namun tidak linier karena *overhead* sinkronisasi dan fraksi serial kode membatasi pertumbuhan performa sesuai Hukum Amdahl, sementara memory bandwidth CPU menjadi *bottleneck* pada skala thread yang lebih tinggi. OpenCL justru memiliki potensi

skalabilitas yang lebih menjanjikan, pada pengujian ini *overhead* transfer data mendominasi karena jumlah partikel relatif kecil dan *hardware* yang digunakan adalah integrated GPU, namun apabila beban kerja ditingkatkan signifikan (1 juta partikel ke atas) pada *discrete* GPU, rasio komputasi terhadap *overhead* transfer akan meningkat drastis sehingga keunggulan arsitektur SIMD GPU dapat termanifestasi secara nyata.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian komparatif yang telah dilakukan pada sistem simulasi partikel dinamis, dapat disimpulkan bahwa penerapan arsitektur komputasi heterogen memberikan hasil performa yang bervariasi tergantung pada karakteristik *hardware* yang digunakan. Mode *Sequential* yang berjalan secara linier pada satu *core* CPU mencatatkan waktu eksekusi paling lambat, yaitu 2.13784 detik. Implementasi *multi-threading* menggunakan OpenMP dengan konfigurasi 2 *threads* terbukti mampu mengoptimalkan utilitas *multi-core* CPU dengan memotong waktu pemrosesan menjadi 1.16987 detik, menghasilkan percepatan (*speedup*) sebesar 1.827x. *Speedup* yang tidak mencapai nilai ideal 2.00x disebabkan oleh *overhead* sinkronisasi *thread*, *memory bandwidth sharing*, serta adanya fraksi serial dalam kode yang tidak dapat diparalelkan sesuai prediksi Hukum Amdahl. Sementara itu, mode OpenCL pada integrated GPU mencatatkan waktu eksekusi 2.18805 detik dengan *speedup* 0.977x, lebih lambat dari *baseline* Sequential. Hasil ini bukan kegagalan implementasi, melainkan konsekuensi dari karakteristik integrated GPU yang berbagi memori dengan CPU, sehingga *overhead transfer* data *host-to-device* mendominasi dan melebihi keuntungan komputasi paralel yang diperoleh. Dari perspektif skalabilitas, OpenMP menunjukkan strong scaling yang terbatas akibat fraksi serial kode dan *memory bandwidth sharing*, sementara OpenCL berpotensi mengungguli CPU secara signifikan pada skala beban kerja yang jauh lebih besar dengan dukungan hardware discrete GPU. Temuan ini menegaskan bahwa keunggulan arsitektur GPU melalui OpenCL baru akan optimal apabila dijalankan pada *discrete* GPU dengan *VRAM dedicated* dan *workload* dengan *compute-to-memory ratio* yang tinggi.

5.2 Saran

Meskipun sistem telah berhasil menunjukkan lonjakan performa yang tinggi, terdapat beberapa ruang pengembangan yang dapat diimplementasikan pada penelitian atau proyek selanjutnya. Pertama, pengujian selanjutnya disarankan dilakukan pada *discrete* GPU dengan *VRAM dedicated* agar keunggulan

arsitektur paralel masif OpenCL dapat dibuktikan secara optimal, sekaligus menggunakan alokasi memori yang lebih efisien seperti pemanfaatan *Local Memory* (LDS) untuk mereduksi *bandwidth bottleneck* saat thread GPU melakukan pembacaan data posisi partikel secara berulang. Kedua, sistem simulasi dapat dikembangkan lebih lanjut dengan menambahkan kalkulasi interaksi antar-partikel secara penuh seperti deteksi tabrakan fisik atau algoritma *Spatial Partitioning* (seperti *Grid-based* atau *Barnes-Hut Tree*) untuk menguji batas ekstrem performa komputasi heterogen pada skenario dunia nyata yang lebih kompleks. Terakhir, visualisasi hasil eksekusi disarankan untuk diintegrasikan secara langsung menggunakan pustaka grafis *real-time* seperti OpenGL atau DirectX melalui mekanisme *Interoperability* dengan OpenCL, sehingga pergerakan ratusan ribu partikel dapat langsung di *render* di layar tanpa memerlukan proses ekspor berkas gambar mentah secara manual.

DAFTAR PUSTAKA

- Afif, A. Z., Safaraz, D. A., Febriyanti, D., Febrianti, R. A., & Aminah, S. (2024). Revolusi teknologi dalam arsitektur komputer: Tantangan dan peluang di era digital. Program Studi Informatika, Fakultas Sains dan Teknologi, UIN Sultan Maulana Hasanuddin Banten.
- Amazon Web Services. (n.d.). What's the difference between a CPU and a GPU? AWS.
<https://aws.amazon.com/id/compare/the-difference-between-gpus-cpus/>
- Lawrence Livermore National Laboratory. (n.d.). OpenMP programming model. LLNL HPC Tutorials.
https://hpc-tutorials.llnl.gov/openmp/programming_model/
- Rakotovao, T., Puschini, D., Mottin, J., Fradet, C., Paindavoine, M., Bensrhair, A., & Laugier, C. (2015). Intelligent vehicle perception: Toward the integration on embedded many-core. In Proceedings of the 6th Workshop on Parallel Programming and Run-Time Management Techniques for Many-Core Architectures (PARMA-DITAM '15). Association for Computing Machinery. <https://doi.org/10.1145/2701310.2701313>
- Rodriguez, D., Gomez, D., Alvarez, D., & Rivera, S. (2023). A review of parallel heterogeneous computing algorithms in power systems. *Heliyon*, 9(10), e21047. <https://doi.org/10.1016/j.heliyon.2023.e21047>
- Rosenberg, J. (2017). Security in embedded systems. In A. Vega, P. Bose, & A. Buyuktosunoglu (Eds.), *Rugged embedded systems* (Chap. 6). Morgan Kaufmann. <https://doi.org/10.1016/B978-0-12-802459-1.00006-3>